

# CS 621 HW2

Ethan Reinhart

October 2025

## 1

### 1.1 Question

Prof. Sharp plans to skate along a highway starting from a point  $A$  and ending with a point  $B$ . The professor carries a water bottle. With the bottle full, Prof. Sharp can skate  $m$  miles before running out of water. There are  $n$  water stops along the highway from which Prof. Sharp can refill the water bottle. Now given the distance  $d_1 < d_2 < \dots < d_n$  of the  $n$  water stops from the starting point  $A$ , where  $d_1 = 0$  is the distance of the starting point  $A$  and  $d_n$  is the distance of the ending point  $B$ , Prof. Sharp wants to find the minimum number of water stops along the highway to refill the bottle such that the bottle never runs out of water. Notice that formally, you are finding a subset  $s_1 < s_2 < \dots < s_k \subseteq 1, 2, \dots, n$  with the minimum cardinality such that  $s_1 = 1$ ,  $s_k = n$ , and  $d_{s_{i+1}} - d_{s_i} \leq m$  for each  $i < k$  (Prof. Sharp shall certainly stop at the starting and ending point).

Suppose that there exists a stopping plan making Prof. Sharp never run out of water, provide a greedy algorithm helping Prof. Sharp find the minimum number of water stops.

### 1.2 Solution

Maximum range per fill  $m$ ; sorted stop distances  $d[1 \dots n]$  with  $d_1 = 0$  ( $A$ ) and  $d_n = B$ ; assume feasibility: every gap  $d[i+1] - d[i] \leq m$ .

```
GreedyRefills(d[1...n], m):
    i = 1                # currently at stop i (start at A)
    plan = [1]           # must include start
    while i < n:
        j = i
        while j+1 <= n and d[j+1] - d[i] <= m:
            j = j+1       # extend to the farthest reachable stop
        if j == i:
```

```

        return -1    # won't happen under assumption
    i = j
    plan.append(i)
return plan          # includes n at the end

```

Time:  $O(n)$

### 1.2.1 Proof

Let  $G = (g_1 = 1 < g_2 < \dots < g_k = n)$  be the greedy plan, and let  $O = (o_1 = 1 < o_2 < \dots < o_t = n)$  be any feasible plan with minimum number of stops (an optimal plan).

We prove by induction that  $g_r \geq o_r$  for all  $r$  (greedy never stops earlier than an optimal plan at the same ordinal stop).

1. Base  $r = 1$ : both are 1.
2. Step: suppose  $g_r \geq o_r$ . From stop  $g_r$ , greedy chooses the farthest reachable stop  $g_{r+1}$  (i.e., the largest  $j$  with  $d[j] - d[g_r] \leq m$ ). Since  $o_{r+1}$  must be reachable from  $o_r$  and  $o_r \leq g_r$ , we have  $d[o_{r+1}] - d[g_r] \leq d[o_{r+1}] - d[o_r] \leq m$ , so  $o_{r+1}$  is also reachable from  $g_r$ . Hence  $g_{r+1} \geq o_{r+1}$  by farthest-reach choice.

## 2

### 2.1 Question

Let  $A_n = a_1, a_2, \dots, a_n$  be a set of distinct coin types (e.g.,  $a_1 = 50$  cents,  $a_2 = 25$  cents,  $a_3 = 10$  cents, etc). Note that  $a_i$  may be any positive integer such that  $a_1 > a_2 > \dots > a_n$  and each type is available in unlimited quantity. Given  $A_n$  and an integer  $C > 0$ , the coin changing problem is to make up the exact amount  $C$  using the minimum total number of coins.

Let  $A_n = k^{n-1}, k^{n-2}, \dots, k^0$  for some integer  $k > 1$ . Provide a greedy algorithm solving the coin changing problem.

### 2.2 Solution

```

GreedyKBaseCoins(C, k, n):          # denominations: k^(n-1) ... k^0
    count[0...n-1] = 0
    for i from n-1 to 0:             # i indexes the exponent
        coin = k^i
        count[i] = C // coin         # integer division
        C = C % coin
    return count                     # total coins = sum count[i]

```

Time:  $O(n)$

### 2.2.1 Proof

Every nonnegative integer  $C$  has a unique base- $k$  expansion:

$$C = \sum_{i=0}^{n-1} b_i k^i, \quad 0 \leq b_i \leq k-1$$

Using  $b_i$  coins of value  $k^i$  yields exactly  $C$  with a total of  $\sum b_i$  coins. That's minimal because

1. If any solution uses greater than  $k$  coins of some denomination  $k^i$ , you can replace  $k$  of them by one coin  $k^{i+1}$ , decreasing the coin count. Repeating, any optimal solution must have digits  $c_i \in 0, \dots, k-1$ .
2. With the digit bounds enforced, the coin set is forced to be the base- $k$  digits  $(b_{n-1}, \dots, b_0)$ . So, the minimal coin count is  $\sum b_i$ .

Let  $g_i$  be the greedy number of  $k^i$  coins. Suppose an optimal solution  $o$  differs at the highest place  $i'$ , where  $o_{i'} < g_{i'}$ . Then the remaining amount in  $o$  at level  $i'$  exceeds what  $o$  can cover using smaller coins without using at least  $k$  coins of some lower level. Exchanging  $k$  coins of  $k^{i'-1}$  for one coin  $k^{i'}$  either matches the greedy choice or reduces the count, contradicting optimality of  $o$ . Therefore, greedy is optimal.